CSE 1321L: Programming and Problem Solving I Lab
Assignment 6 – 100 points
Methods, Searching and Sorting Information

What students will learn:

1) Searching for information in arrays
2) Sorting arrays to make the search process more efficient
3) Designing and calling methods

Overview: Arrays are great, but just archiving data haphazardly in an array is not enough. For most large applications today, you need to be able to quickly find, update, and use that information. For instance, when you visit our class section in D2L, the server has to find your name, grades, and assignment submissions from a database of all students at Kennesaw State. This assignment will help you practice the foundational skills you'll need to one day develop these large scale systems.

Assignment 6A: *Benchmark Test for Searching.* In the lecture class, we have repeatedly claimed that binary search is faster than linear search on a sorted array. But don't take our word for it – let's try to prove it by comparing the two!

In this lab, you will create an array of size **one million** (1,000,019). Fill it in reverse sequentially with values (e.g. array[0] should equal 1,000,018, array[1] should equal 1,000,017, etc). Then the computer should select a random target number between 0 and array size – 1. You should then search the array for this value twice – once with a linear search algorithm, and then with a binary search algorithm. **Start searching from the end of the array, rather than the beginning**. Keep track of how long each one took, using the following criteria:

- Linear Search: Number of loop iterations before the target number was found
- Binary Search: Number of midpoints chosen (a.k.a "guesses") taken before the target number was found

Once both algorithms have been run, display the results and print which algorithm found the number first (or if there was a tie).

Sample Output #1:

```
[Linear Vs. Binary Search]
The target value is 307620
Linear Search: 307621 loop(s)
Binary Search: 20 guess(es)
Binary Search is faster this time!
```

Sample Output #2:

```
[Linear Vs. Binary Search]
The target value is 1
Linear Search: 2 loop(s)
Binary Search: 19 guess(es)
Linear Search is faster this time!
```

In the rest of this assignment, you're going to create your own methods and call them from main. We haven't talked about classes and Object-Oriented Programming (OOP) yet, so we're going to have to "break the rules" a bit. For the Java and C# folks, this means adding a special keyword before each method you write – called **static**. In simple terms, this means that we don't want to use OOP (though there's much, much more to it).

Assignment 6B: *Random Sort.* In lecture, we told you that Bubble Sort, Selection Sort, and Insertion Sort have terrible worst case time complexities of O(n). However, there is an algorithm with even worst time complexity – Random Sort! Random Sort works by randomly shuffling the elements in the array and then checking to see if they're in non-decreasing order. If they aren't, we repeat the process again. This results in hypothetical worst case time complexity of O(∞); in other words, it might run FOREVER!

To begin, create a 1D integer array of size 17. Fill each index with a random value ranging from 1 to 359 inclusive. You will then design and implement the Random Sort algorithm using the following methods:

- Create a method called **check_if_sorted ()**. It should take in a 1D integer array and return a boolean value. It should return TRUE if the array is sorted in non-descending order, and FALSE otherwise.
  Hint: If you compare elements in the array and a pair is in the wrong order, that would mean the array is not in non-descending order.

- Create a method called **shuffleArray ()**. It should take in a 1D integer array and return a 1D integer array. Shuffle the array so that the values are in random different indexes, and return altered array.
  Hint: There are many approaches to solve this problem – making a second array in the shuffleArray () method might be part of the answer.

- Create a method called PrintArray (). It should take in a 1D integer array and return nothing. Simply print the current values of the array when it's called.

Sample Output:

```
[Random Sort]

Printing array...
2, 9, 232, 1, 89, 74, 34, 122, 344, 19, 1, 1, 2, 78, 43, 12, 19,
Not sorted yet!

Shuffling array...

Printing array...
19, 2, 9, 34, 232, 1, 19, 344, 43, 89, 74, 122, 1, 2, 1, 78, 12,
Not sorted yet!

Shuffling array...
//And so on… (Not part of the code)
```

```
Printing array...
1,  1, 1, 2, 2, 9, 12, 19, 19, 34, 43, 74, 78, 89, 122, 232, 344,

Hooray, it's sorted! And it only took 987456 attempts!
//You will get a different number each time
```

Assignment 6C: *Overloaded Functions.* When we create overloaded functions, they should produce similar output to the original. For this assignment, we're going to break that rule! You will create a function called **surprise_function ()** which takes in two integers and adds them together. You will then create the following overloaded versions, which should do different things based on the data type passed in:

Data Type	**Operation**
Integer	Addition
Float	Division
Double	Modulus
Char	Concatenation (Return as String)
Boolean	Result of AND

You should prompt the user for what data type they want to enter, then ask for two values of that data type. Then, call surprise_function (), pass in the values, store the result in an appropriate variable, then print the variable.

Note: You must make overloaded functions for this assignment – they must all be called **surprise_function ()**. You can not create unique, non-overloaded functions like surprise_function_booleans().

Sample Output #1:

```
[Overloaded Surprise Function]
What data type do you want to enter? float
Value #1: 2.3
Value #2: 10.2
Calling surprise_function()...
The result is 0.22549
```

Sample Output #2:

```
[Overloaded Surprise Function]
What data type do you want to enter? boolean
Value #1: true
Value #2: false
Calling surprise_function()...
The result is false
```

Sample Output #2:

```
[Overloaded Surprise Function]
What data type do you want to enter? char
Value #1: k
Value #2: s
```

```
Calling surprise_function()...
The result is ks
```

Submission:

1. You will submit 3 separate files
2. File names and class names must be correct.
3. Upload all files (simultaneously) to the assignment submission folder in Gradescope.

Appendex: Dynamic Arrays for C++

There are several popular compilers, but there are a few key differences between what they will and won't allow you to do in C++. In some compilers, the following code is perfectly fine:

```
int i = 12;
int array[i];
```

Other compilers will throw an error because you are not using a constant to define the size of the array. This limitation obviously poses a challenge to completing certain assignments. If your compiler does not allow you to use variables to define arrays, use the following workarounds:

For 1D Arrays

```
int x = 12;
int * array = new int[x];
```

For 2D Arrays

```
int x = 12;
int y = 24;
int ** array = new int * [x];
for(int i = 0; i < x; i++){
   array[i] = new int[y];
}
```

Once you have done so, you can use the arrays as normal (e.g. array[0][0] = 34 will assign the array at index 0, 0 to the value of 34). In our data structures class we discuss more about **why** this works, but for now, it is a useful workaround if your compiler does not allow you to use variables to create arrays from user input.